# Step 1 — Baseline: misurare tutto prima di addestrare

## Obiettivo

Sapere esattamente quanto valgono Winnow-12B e JPT-9B **prima** del fine-tuning, sugli split congelati allo Step 0, con la migliore configurazione ottenibile **senza** cambiare i pesi. Ogni guadagno del fine-tuning si misura contro questa linea, con lo stesso protocollo.

Senza questo step non si può dire se il fine-tuning ha migliorato il modello o se lo stesso risultato si otteneva cambiando solo soglia, temperatura o domanda.

## Prerequisito

Gli split di Step 0 devono essere congelati (`data/finetune/splits/manifest.json`). La baseline usa **solo** `dev_select` e `dev_confirm` per le scelte; i test (test_id, test_ood, AggreFact) si eseguono per la baseline e poi non si riguardano fino alla valutazione finale del modello fine-tunato.

## Input e output

| | |
|---|---|
| Input | modelli di partenza; split di Step 0; test di LLM-AggreFact (cache già presenti in `results/runs/<modello>/`); benchmark di oblio |
| Output | `results/runs/baseline_<modello>.json` (da creare): configurazione scelta, metriche per blocco e per fonte, test comportamentali, oblio |

## Cosa si può cambiare senza addestrare

| Leva | Cosa cambia | Esempio |
|---|---|---|
| **Soglia** | sopra quale probabilità si dice "supportato" | 0,30 (fittata su Jev) contro una soglia propria del modello |
| **Temperatura** | quanto sono "schiacciate" le probabilità: `softmax(z / T)` | Winnow T = 1; JPT T = 1,087; si può rifittare |
| **Insieme di domande** | una domanda sola o la regola a tre domande `min(p1, 1−p2, 1−p3)` | solo `support_simple` contro la regola di Jev |

**Esempio di cosa fa la temperatura.** Logit delle lettere [z_falso, z_vero] = [0,2, 2,0]:

- T = 1: P(vero) = 1 / (1 + e^(−1,8)) = 0,86
- T = 2: P(vero) = 1 / (1 + e^(−0,9)) = 0,71

La decisione a soglia 0,5 non cambia, ma le probabilità diventano meno sicure. Se il modello è sovra-confidente (ECE alta), una T > 1 lo ricalibra.

## Procedura

### 1.1 Inferenza sul nostro dev

Per ciascun modello, le tre domande della regola di Jev su `dev_select` e `dev_confirm` (≈ 7.000 righe in tutto, poche ore su L40S). I benchmark della repo accettano un file di dati con `--data`; vanno estesi per leggere gli split di Step 0 e scrivere le cache in una cartella per split.

### 1.2 Scelte, tutte su `dev_select`

| ID | Cosa | Costo |
|---|---|---|
| B1 | soglia fittata (BAcc macro sulle fonti, griglia 0,01–0,99) | 0, dalla cache |
| B2 | temperatura (minimo log-loss) + soglia | 0, dalla cache |
| B3 | solo `support_simple`, con soglia e temperatura proprie | 0, dalla cache |

La configurazione migliore fra B1–B3 sul `dev_select` si conferma una volta su `dev_confirm` (la BAcc non deve scendere di più di ~1 punto). Quella è la **baseline vera** del modello.

Esempio di lettura di B3: se con la sola `support_simple` JPT arriva a 0,2 punti dalla regola a tre domande, la regola vale poco e costa 3× in inferenza. Questo informa la scelta S1/S3 del training (03_rendering_formato.md).

### 1.3 Test della baseline (una volta)

Con la configurazione congelata:

| Blocco | Come | Note |
|---|---|---|
| test_id | inferenza sugli split di test delle fonti di training | |
| test_ood | inferenza su SciFact, Climate-FEVER, HealthVer/COVID-Fact, HaluBench, HalluMix | |
| AggreFact test | **nessuna nuova inferenza**: le cache esistono già, si applicano soglia e temperatura scelte sul nostro dev (come fa `calibrate_transfer.py apply`) | risultato **zero-shot** |
| AggreFact test, secondario | soglia fittata sul dev di AggreFact (già calcolata: Winnow-BF16 77,42, JPT-9B 76,71) | solo per confronto con i numeri esistenti, dichiarato |
| Test comportamentali | generati da `dev_select` (06_valutazione.md, §4) | |
| Oblio | JevBench pubblico, Kev-v9 clean, typed-decisions test, con i loro script ufficiali | |

Per Winnow-BF16 i numeri di oblio sono già pubblicati dagli autori: **JevBench 85,28%, Kev-v9 clean 81,45%, typed-decisions 70,20%**. Per JPT vanno misurati.

### 1.4 I modelli base (per il controllo C1)

Stesso prompt e stessa procedura su **Gemma-4-12B** (base di Winnow) e **Qwen3.5-9B** (base di JPT). Servono solo per il confronto finale: se dopo il fine-tuning il modello base arriva allo stesso punto del decision model, il vantaggio sta nei dati, non nell'inizializzazione.

## Tabella di baseline (formato atteso)

| Blocco | Metrica | Winnow-BF16 | JPT-9B |
|---|---|---|---|
| dev_select | BAcc macro | *da misurare* | *da misurare* |
| test_id | BAcc macro | *da misurare* | *da misurare* |
| test_ood | BAcc macro | *da misurare* | *da misurare* |

| Blocco | Metrica | Winnow-BF16 | JPT-9B |
|---|---|---|---|
| AggreFact test (zero-shot, soglia dal nostro dev) | BAcc macro / AUC macro | *da misurare* / 85,02 | *da misurare* / 84,57 |
| AggreFact test (secondario, soglia dal dev di AggreFact) | BAcc macro | 77,42 | 76,71 |
| Test "solo claim" | P(supportato) media | *da misurare* | *da misurare* |
| Oblio | JevBench / Kev-v9 / typed | 85,28 / 81,45 / 70,20 | *da misurare* |
| Configurazione | soglia / T / domande | *da dev_select* | *da dev_select* |

L'AUC su AggreFact è già nota perché non dipende dalla soglia.

## Controlli

- Soglia e temperatura si fittano **solo** su `dev_select`, mai su un test né sul dev di AggreFact (tranne la soglia secondaria dichiarata).
- La configurazione scelta si conferma su `dev_confirm`.
- Le cache sono legate all'identità del modello, del prompt e dei dati: un cambiamento produce una cache nuova, non si mescola.

## Criteri per passare allo step successivo

- Tabella di baseline completa per entrambi i modelli.
- Configurazione di partenza congelata (soglia, temperatura, domande) per ciascun modello.

## Errori comuni

- Confrontare il modello fine-tunato con la baseline a soglia 0,30: quella soglia è stata fittata su Jev e regalerebbe al fine-tuning un guadagno finto.
- Confrontare il fine-tunato zero-shot con la baseline a soglia fittata sul dev di AggreFact (o viceversa): baseline e fine-tunato vanno confrontati con la stessa procedura.
- Rieseguire i test della baseline più volte mentre si sperimenta: i test si guardano una volta per la baseline e una volta per il modello finale.